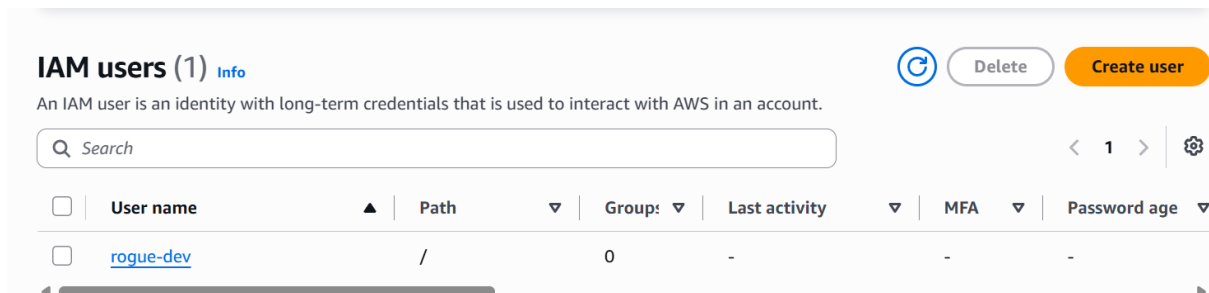


AWS project write up

Automated Secret Scanner & Auto-Remediation Bot

Automating incident response by disabling compromised AWS IAM access keys with Lambda



rogue-dev doesn't need to actually *do* anything in AWS it just needs to exist so your bot can search for its access key and deactivate it. Leaving it with zero permissions is a security best practice

Rogue-dev: AKIAYDZUOO434V5T57W4

Detailed Execution Sequence

1. Trigger & Event Ingestion

- The workflow receives an execution payload or trigger containing a compromised `target_key_id` (e.g., AKIA...).
- *In a production setup, this event can be dispatched automatically via Amazon EventBridge, AWS GuardDuty alerts, or security log triggers.*

```
PS C:\Users\LethaboSangweni\aws-secret-remediation-bot> aws configure
AWS Access Key ID [*****EI05]:
AWS Secret Access Key [*****HSjY]:
Default region name [us-east-1]:
Default output format [None]:
PS C:\Users\LethaboSangweni\aws-secret-remediation-bot> python remediate_key.py
Enter Access Key ID to test deactivation: AKIAYDZUOO434V5T57W4
[i] Searching owner for key: AKIAYDZUOO434V5T57W4...
[+] Key found! Owned by user: 'rogue-dev'
[SUCCESS] Access key AKIAYDZUOO434V5T57W4 has been deactivated!
PS C:\Users\LethaboSangweni\aws-secret-remediation-bot> notepad lambda_function.py
PS C:\Users\LethaboSangweni\aws-secret-remediation-bot> |
```

2. IAM Query & Account Traversal (boto3)

- The Lambda function initializes an IAM client using the boto3 SDK under a least-privilege execution role.
- User Enumeration: Uses IAM paginators (`iam.get_paginator('list_users')`) to handle accounts of any size without hitches or pagination caps.
- Key Inspection: For every IAM user in the account, the function invokes `iam.list_access_keys` to inspect active and inactive Access Key IDs attached to that user.

3. Target Identification & Remediation

- Matching: Once a match for the target `AccessKeyId` is identified, the function correlates the key directly to its owning IAM user.
- Containment: The function immediately issues an `iam.update_access_key` API call, setting the key status from Active to Inactive.

4. Status Logging & Return Payload

- The function logs the remediation event (User ID, Key ID, and execution timestamp) to Amazon CloudWatch.
- Returns a structured 200 OK JSON response confirming successful deactivation, or a 404 Not Found if the key does not exist in the environment.

Principle of Least Privilege

To enforce the Principle of Least Privilege (PoLP), the Lambda function operates under a dedicated IAM role attached to a scoped policy (KeyRemediatorPolicy.json). The role grants *only* the specific API actions required to discover and disable the compromised access key without granting broad administrative rights (iam:*) or destructive privileges like iam>DeleteAccessKey or iam>DeleteUser

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- Roles
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer

Summary

Creation date: July 28, 2026, 19:11 (UTC+02:00)

Last activity: 6 minutes ago

ARN: arn:aws:iam::557918877495:role/remediate_key-role-g5bezs5w

Maximum session duration: 1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) Info

You can attach up to 10 managed policies.

Filter by Type: All types

Policy name	Type	Attached entities
AWSLambdaBasicExecutionRole	AWS managed	1
KeyRemediatorPolicy	Customer managed	1

Policy details

Type
Customer managed

Creation time
July 28, 2026, 18:28 (UTC+02:00)

Edited time
July 28, 2026, 18:28 (UTC+02:00)

ARN
arn:aws:iam::557918877495:policy/KeyRemediatorPolicy

Permissions

Entities attached

Tags

Policy versions (1)

Last Accessed

Permissions defined in this policy

Edit

Summary

JSON

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

Search

Allow (2 of 472 services)

Show remaining 470 services

Service	Access level	Resource	Request condition
CloudWatch Logs	Limited: Write	region string like All	None
IAM	Limited: List, Write	All resources	None

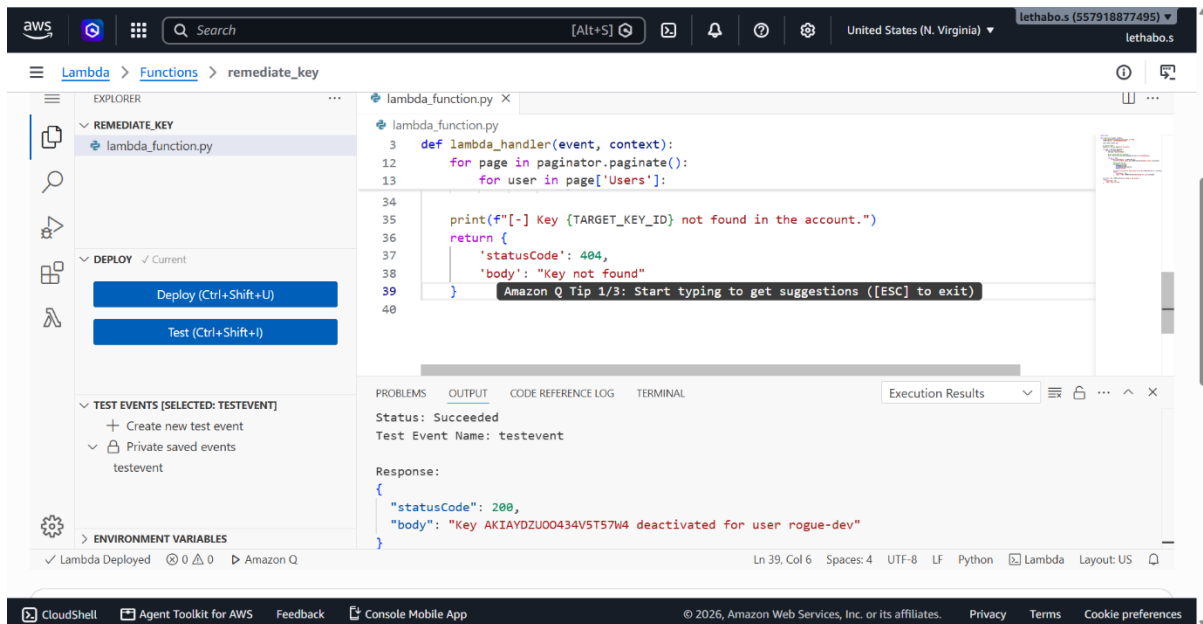
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "IAMKeyRemediationPermissions",
      "Effect": "Allow",
      "Action": [
        "iam:ListUsers",
        "iam:ListAccessKeys",
        "iam:UpdateAccessKey"
      ],
      "Resource": "*"
    }
  ]
}
```

Testing & Verification

1. **Provision Test User:** Create a temporary IAM user (rogue-dev) with zero permissions and generate an active access key (e.g., AKIAYDZUOO434V5T57W4).
2. **Execute Test:** Trigger the Lambda function using a test payload containing the target key:

JSON

```
{
  "target_key_id": "AKIAYDZUOO434V5T57W4"
}
```



3. **Verify Remediation:** Confirm the function returns a 200 OK response and verify in the IAM Console under **rogue-dev > Security credentials** that the key status has automatically changed from **Active** to **Inactive**.

Future Enhancements & Production Roadmap

- **Zero-Touch Automation (EventBridge & GuardDuty):** Trigger the Lambda function automatically when AWS GuardDuty detects compromised credentials or exposed keys in CloudTrail logs.
- **Real-Time Security Notifications (SNS & Slack):** Send instant alerts to Slack webhooks or email via Amazon SNS when a key is contained (e.g., “Key AKIA... for user rogue-dev was automatically set to Inactive”).
- **Multi-Account IR (AWS Organizations):** Update the boto3 script to use sts:AssumeRole so a single security account can remediate leaked keys across all member AWS accounts.
- **Key Rotation Workflows:** Automatically generate a replacement key via AWS Secrets Manager and securely notify the impacted developer.